# 1. Purpose and Scope

### 1.1 Purpose

This document provides guidance for the Independent Verification and Validation (IV&V) team on organizing, planning, executing, and tracking testing activities using Jira and Xray.

It establishes a consistent process for linking testing work to the appropriate application and release, reusing test cases, recording results and evidence, and tracking defects through retesting.

### 1.2 Intended Audience

This guide is intended for IV&V testers and application lead testers who manage or perform testing activities in Jira and Xray.

### 1.3 Scope

This process applies when the IV&V team performs testing for an application release or conducts additional verification and validation activities.

Depending on the testing needs, activities may include functional testing, regression, automated UI testing, performance testing, accessibility testing, or other agreed validation activities.

The guide covers:

- Creating Test Plans and selecting reusable test cases.
- Creating and organizing Test Executions.
- Recording test results, supporting evidence, and defects.
- Retesting fixes and reporting the testing outcome.

The guide explains how to manage these activities in Jira and Xray. Detailed instructions for developing or running Selenium, Playwright, LRE, or other testing tools are outside its scope.

# 2. IV&V Jira and Xray Structure

The IV&V team uses a shared Jira project with Xray to organize test cases, identify applications, track releases, and manage testing activities. The following areas support this structure.

### 2.1 Xray Test Repository

The Xray Test Repository is the central location where the IV&V team stores and organizes reusable test cases. Test cases are grouped into folders named after the applications they support.

This organization is similar to the application folders used in the ALM Test Plan area. Testers open the appropriate application folder to locate and review the test cases needed for their testing activities.

The repository holds the reusable test definitions. Test Plans and Test Executions reference these test cases to organize testing scope and record results.

*[Screenshot here: Xray Test Repository showing application folders and the test cases within a selected folder.]*

### 2.2 Jira Components

The Components tab lists the applications supported by the IV&V team. Each application entry includes:

- The application name.
- A brief description of the application.
- The IV&V application lead tester, recorded as the Component lead.

When creating a Jira issue for testing work, the tester selects the appropriate application in the Component/s field. This associates the issue with the application and helps the team organize, filter, and report on its work.

The IV&V tester is responsible for ensuring that the application is listed in Components. If it is missing, the tester adds the application name, description, and lead tester. If the tester does not have permission to create Components, they coordinate with the Jira project administrator to have it added.

The repository folder and Jira Component should use consistent application names. They are separate records; placing a test case in an application folder does not automatically assign its Component.

*[Screenshot here: Components tab showing application names, descriptions, and Component leads.]*

### 2.3 Jira Releases

The Releases tab contains the application versions used to organize the team's testing work.

The application test lead is responsible for creating the release entry when a new application release is announced. Before creating an entry, the lead checks whether it already exists to avoid duplication.

Each release entry should identify:

- The application and release number, such as CIS 1.1.
- The start of the testing window.
- The planned end of the testing window.
- A description of the release and relevant testing information.

Jira normally labels the version dates Start date and Release date. In the IV&V project, where these dates represent the testing window, the description should clearly state that Release date represents the planned testing end, rather than the application's production deployment date.

When creating a Test Plan, Test Execution, or other applicable testing issue, the tester selects this release in the Fix Version/s dropdown. The Component identifies the application, while Fix Version/s identifies the release being tested.

*[Screenshot here: Releases tab showing an application release, its dates, and description.]*

*[Screenshot here: Jira issue showing the Component/s and Fix Version/s fields.]*

### 2.4 IVV Demo Board

The IV&V team uses the Jira board named IVV Demo to track assigned work and testing progress, including Test Plans and Test Executions.

Team members use the board to review their assignments, monitor the status of testing activities, and identify work that requires attention. Testers keep their assigned issues current so the board reflects the progress of the work.

The IVV Demo board also allows the team to filter issues by each tester's name. Select a tester's name to view their assigned issues, or clear the filter to return to the broader board view.

The board displays issues according to its configured filters and workflow columns. Detailed test results, screenshots, and linked defects are recorded within the associated Xray Test Runs. The board's issue status and a Test Run's pass/fail result represent different information.

*[Screenshot here: IVV Demo board showing its workflow columns, representative Test Plan and Test Execution issues, and the tester-name filter.]*

# 3. Preparing for Testing

When the IV&V team performs testing for an application release or additional verification and validation, the assigned tester confirms the testing scope and checks that the necessary information and access are available.

### 3.1 Confirm the Testing Scope and Window

Review the information provided through email, meetings, or other team communications. Confirm:

- **Application and release:** Which application and version will be tested?
- **Testing scope:** Which features or business processes are included or excluded?
- **Testing type:** Functional regression, automated UI, performance, accessibility, or other testing.
- **Testing window:** When testing can begin and when it must be completed.
- **Assigned tester:** Who will perform the testing and coordinate questions or findings?

If any information is unclear, the tester coordinates with the program owner before proceeding.

### 3.2 Verify Testing Readiness

Before starting testing, confirm that:

- The application is listed under Components.
- The appropriate release is available under Releases for selection in Fix Version/s. If it is missing, coordinate with the application test lead.
- The test environment, user accounts, permissions, and test data are available.
- Any required browsers or testing tools are available.
- Relevant test cases can be located in the application's Xray Test Repository folder. Identify any cases that need to be created or updated.

Record any missing prerequisites or blockers so the team can address them.

### 3.3 Proceed to Test Planning

Once the scope and prerequisites are understood, the tester creates a Test Plan to organize the testing approach and selected test cases for the application release.

The next section will explain what to include in the Test Plan and how it connects to Test Executions.

# 4. Creating a Test Plan

### 4.1 Purpose of the Test Plan

For the IV&V team's process, create a Test Plan for each application release or distinct testing effort. The Test Plan identifies the testing scope, describes the approach, and brings together the selected test cases and associated Test Executions.

The testing approach may differ between releases. For example, one release may require functional regression testing, while another may require performance or accessibility testing.

- **Test Plan:** What we intend to test and how we will approach it.
- **Test Execution:** A specific testing run where results and evidence are recorded.
- **Test cases:** The individual checks and expected results.

### 4.2 Enter the Test Plan Details

When creating the Test Plan, complete the following information:

- **Summary:** Use a descriptive name, such as CIS 1.1 – Functional Regression Test Plan.
- **Component/s:** Select the application, such as CIS.
- **Fix Version/s:** Select the release being tested, such as CIS 1.1.
- **Assignee:** Select the IV&V tester responsible for coordinating the plan.
- **Description:** Document the testing scope and approach using the guidance below.

*[Screenshot here: Create Test Plan screen showing Summary, Component/s, Fix Version/s, Assignee, and Description.]*

### 4.3 Describe the Testing Scope and Approach

The Description should provide enough information for another IV&V team member to understand the planned testing. Include the following, as applicable:

| Item | What to document |
|---|---|
| Testing objective | What the testing is intended to verify or validate. |
| Testing type | Functional, regression, performance, accessibility, or other testing. |
| Execution approach and tools | Whether testing is manual, automated, or a combination. Identify tools such as Selenium, Playwright, LRE, or JMeter when applicable. |
| Environment and browsers | The test environment and required browsers, such as Edge or Firefox. |
| Testing window | The planned start and end dates. |
| Prerequisites | Required access, accounts, test data, environment readiness, or other dependencies. |

Testing type and execution approach are separate choices. For example, functional regression testing can be performed manually or through automation.

Detailed test steps belong in the test cases; they do not need to be repeated in the Test Plan Description.

*[Screenshot here: An example Test Plan Description containing the testing scope and approach.]*

### 4.4 Add the Relevant Test Cases

Review the test cases in the application's Xray Test Repository folder and add those relevant to the agreed scope.

Do not automatically include every test case simply because it belongs to the application. Confirm that the selected cases are applicable to the release and update or create cases where coverage is missing.

*[Screenshot here: Test Plan showing the selected test cases.]*

### 4.5 Organize the Test Executions

Associate the Test Executions with the Test Plan so the team can review progress for the overall testing effort.

A plan may have one or more Test Executions. Separate executions may be useful for different browsers, environments, testing types, or testing cycles.

The next section will explain how to create a Test Execution, associate it with the plan, and prepare it for testing.

# 5. Creating a Test Execution

### 5.1 Purpose of the Test Execution

A Test Execution organizes the test cases to be performed during a specific testing run. It provides the context for recording results, supporting evidence, and associated defects.

For the IV&V team's process, associate each Test Execution with the appropriate Test Plan.

### 5.2 Create the Test Execution from the Test Plan

1. Open the Test Plan for the application and release.
2. Review the included test cases and identify those needed for this execution.
3. Use the available Xray action to create a Test Execution for the selected tests.
4. Complete the Test Execution details described below.
5. Save the issue and confirm that it is associated with the correct Test Plan.

*The exact action names and screen layout may vary with the team's Jira/Xray configuration.*

*[Screenshot here: Test Plan showing the test selection and action used to create a Test Execution.]*

### 5.3 Enter the Test Execution Details

Complete or verify the following fields. Do not assume that all information is copied automatically from the Test Plan.

| Field | What to enter |
|---|---|
| Summary | A descriptive name, such as CIS 1.1 – Functional Regression – Edge – Cycle 1. |
| Component/s | The application being tested, such as CIS. |
| Fix Version/s | The release being tested, such as CIS 1.1. |
| Assignee | The IV&V tester responsible for the execution. |
| Description | A brief explanation of this testing run, including relevant environment, browser, build, and execution instructions. |

Where the project provides a dedicated Test Environments field, use the team's agreed environment values. Include additional details in the Description as needed.

*[Screenshot here: Test Execution showing Summary, Component/s, Fix Version/s, Assignee, and Description.]*

### 5.4 Verify the Included Test Cases

Before beginning testing:

- Confirm that the execution contains the intended test cases.
- Check that the test steps and expected results are current.
- Confirm that the required accounts, access, and test data are available.
- Verify who will execute each test if more than one tester is involved.
- Confirm that the execution appears in the associated Test Plan.

Adding test cases prepares the execution; it does not mean those tests have been performed.

*[Screenshot here: Test Execution showing its included tests and associated Test Plan.]*

### 5.5 Use Separate Executions When Needed

For the IV&V process, create separate Test Executions when results need to be tracked independently—for example, for different browsers, environments, or testing cycles.

For CIS 1.1, the same functional regression Test Plan could have:

- CIS 1.1 – Functional Regression – Edge – Cycle 1
- CIS 1.1 – Functional Regression – Firefox – Cycle 1
- CIS 1.1 – Defect Retest – Cycle 2

Each execution should include only the test cases needed for that run. For a new retest cycle, create a separate execution to preserve the earlier results and evidence.

The next section will explain how to execute the tests, record pass/fail results, attach screenshots, and report defects.

# 6. Executing Tests and Recording Results

### 6.1 Open the Test Run

1. Open the Test Execution for the correct application, release, and testing cycle.
2. Locate the test case to be performed.
3. Select the available execution action to open its Test Run.
4. Confirm that you are testing in the environment and browser identified for this execution.

A Test Run records the results of an individual test within a Test Execution. Results, evidence, and defects should be recorded against that run. Xray Test Runs guidance

*[Screenshot here: Test Execution showing the included tests and the action used to open a Test Run.]*

### 6.2 Perform the Test Steps

For manual testing, follow the test steps in order and compare the application's actual behavior with each expected result.

Before performing a step, confirm that any required prerequisites and test data are available. Record observations that help explain the outcome, particularly when the actual result differs from the expected result.

For testing performed using external tools, such as Selenium, Playwright, LRE, or JMeter, review the tool results before manually recording the corresponding outcome in Xray. Attach or reference the relevant report as supporting evidence.

*[Screenshot here: Manual Test Run showing test steps, test data, and expected results.]*

### 6.3 Record the Test Results

For the IV&V process, record results as testing progresses.

| Result | When to use it |
|---|---|
| Pass | The performed step or test meets its expected result. |
| Fail | The performed step or test does not meet its expected result. |
| Not completed | Testing has not been performed or cannot be completed because of a blocker. Use the appropriate available status and explain the reason. |

“Not completed” describes the situation; it is not necessarily a status label in the team’s Xray configuration.

For manual tests, record the applicable step results and review the overall Test Run result for accuracy. Do not mark a test as passed if required steps remain unperformed.

If testing is blocked by missing access, unavailable test data, or an environment issue, document the blocker and notify the appropriate contact. Do not treat an unperformed check as a confirmed application failure.

*[Screenshot here: Test Run showing step-level results and the overall test result.]*

### 6.4 Attach Screenshots and Supporting Evidence

When a manual test step fails, attach a screenshot to that failed step and add a brief explanation of the actual result. Xray supports evidence and comments during test execution. Xray execution guidance

For the IV&V team, the explanation should identify:

- What was expected.
- What actually happened.
- Any relevant error message.
- The environment, browser, or test data details needed to understand the failure.

Include logs or tool reports when they help explain the issue. For tests without individual steps, attach the evidence at the Test Run level.

Review attachments before uploading them to avoid exposing passwords, personal information, or other restricted data.

*[Screenshot here: Failed test step with a screenshot attachment and an explanation of the actual result.]*

### 6.5 Create or Link a Defect

When a failure indicates a defect:

1. Check whether the same issue has already been reported.
2. If an existing defect covers the issue, link it to the failed step or Test Run.
3. Otherwise, use the available Xray defect action to create a Bug or the team's configured defect issue type.
4. Include enough information to reproduce the problem, including expected and actual results and supporting evidence.
5. Confirm that the defect is associated with the relevant failed step or Test Run.

Marking a test as failed does not replace reporting a defect. Xray supports both creating defects and associating existing defects with a Test Run or manual test step. Xray defect guidance

Detailed defect reporting and retesting guidance will be covered in Section 7.

*[Screenshot here: Failed test step showing the actions to create or link a defect and the associated defect key.]*

### 6.6 Review and Update Execution Progress

Before finishing a testing session:

- Confirm that results reflect the testing actually performed.
- Check that failed steps have the required screenshots and explanations.
- Verify that identified defects are linked to the relevant results.
- Document unfinished or blocked tests.
- Update the Test Execution's Jira issue status to reflect the current stage of work on the IVV Demo board.

The Jira issue status tracks workflow progress; the Xray Test Run result records the testing outcome. Completing the testing activity does not mean that every test passed.

*[Screenshot here: Test Execution showing recorded results, linked defects, and the Jira issue status.]*

# 7. Defect Follow-Up and Testing Completion

### 7.1 Review and Communicate Defects

For defects identified during testing, confirm that the report includes:

- A clear summary of the problem.
- The application, affected release or build, environment, and browser, as applicable.
- Steps to reproduce the issue.
- Expected and actual results.
- Supporting screenshots or logs and an association with the relevant Test Run.

Coordinate with the program owner or designated application contact to communicate findings and clarify any questions. Keep relevant updates in the defect record so the IV&V team can follow its progress.

*[Screenshot here: Defect showing its description, supporting evidence, and associated testing information.]*

### 7.2 Retest Available Fixes

When a fix is ready for IV&V verification:

1. Confirm which build and environment contain the fix.
2. Create a separate retest Test Execution under the appropriate Test Plan, as described in Section 5.
3. Include the affected test cases and any additional checks needed to assess related functionality.
4. Perform the tests and record the new results and evidence.
5. Reference the original defect in the retest results and add the retest outcome to the defect record.

If the fix works as expected, update the defect according to the team's agreed workflow and your permissions. If the problem remains, document the failure and notify the appropriate contact.

Preserve the original failed results and evidence. Do not replace them with the retest outcome.

*[Screenshot here: Retest execution showing the new result and the associated defect.]*

### 7.3 Summarize the Testing Outcome

At the end of the testing window or when the planned testing is finished, add a brief outcome summary to the Test Plan.

Include:

- The application and release tested.
- The testing performed and the relevant environments or browsers.
- A summary of passed, failed, and unperformed tests.
- Outstanding defects, blockers, and any follow-up testing needed.
- Links to the relevant Test Executions and supporting reports.

When multiple testing cycles exist, distinguish the latest results from earlier runs so repeated executions are not mistaken for additional test coverage.

Share the summary with the program owner and the IV&V team through the agreed communication channel.

*[Screenshot here: Test Plan containing a testing outcome summary and links to Test Executions.]*

### 7.4 Update the Testing Work Status

Update the Test Plan and Test Execution issue statuses according to the team's workflow and the work actually completed.

If testing ends with unresolved defects or unperformed tests, make those limitations clear in the summary. Identify any agreed follow-up work and who will coordinate it.

**Completion of IV&V testing does not mean that all tests passed or that the release has been approved for deployment.** The summary communicates the testing findings; release decisions remain with the authorized stakeholders.

# 8. Maintaining Test Cases and Handling Archived Tests

### 8.1 Keep Test Cases Current

Review existing test cases before reusing them for a new release or testing effort.

- Update steps, test data references, and expected results when application behavior has changed.
- Create new test cases when existing cases do not cover the required functionality.
- Check for existing coverage before creating a new case to avoid unnecessary duplicates.
- Keep test cases in the appropriate application folder in the Xray Test Repository, using clear, descriptive names.

Update reusable test cases to reflect agreed application behavior—not simply to make a failing test pass. Preserve historical execution results and supporting evidence.

*[Screenshot here: Application folder in the Test Repository showing organized test cases.]*

### 8.2 Locate Missing or Archived Tests

If a required test case cannot be found:

1. Confirm that you are searching the correct Jira project and application folder.
2. Check whether search filters or access permissions are hiding the test.
3. Search by the test's issue key or name, if known.
4. If the test remains unavailable or appears archived, contact the Jira/Xray administrator or designated support team.

Include the application name, test key or name, and why the test is needed. Ask support to confirm whether it was archived, moved, or made unavailable for another reason.

Do not assume that a missing test was deleted, and avoid immediately creating a duplicate.

*[Screenshot here: Archived-test indicator or an example support request for a missing test.]*

### 8.3 Review Tests Returned to Use

Restoration options and permissions depend on the organization's Jira/Xray configuration and archival policy. The administrator or support team should confirm whether the test can be restored and who is authorized to restore it.

Once the test is available:

- Confirm that it belongs to the correct application folder.
- Review its steps and expected results for the current application version.
- Update it where necessary before adding it to a new Test Plan or Test Execution.

If restoration is not possible, coordinate with the IV&V application lead tester to determine whether a replacement test case is needed.